

JavaScript

Callbacks & Higher-Order Functions

Frontend - Tag 16 · Lernzettel

Lernziel

Du verstehst, wie Funktionen an andere Funktionen übergeben werden, warum Array-Methoden Callbacks erwarten und wann du `map`, `filter`, `some`, `find` oder `forEach` einsetzt.

Nach diesem Lernzettel kannst du ...

- Funktionen als Werte behandeln und korrekt als Callback übergeben.
- anonyme Funktionen und Arrow Functions als Callbacks schreiben.
- Higher-Order Functions erkennen und Methoden sinnvoll verketteten.
- `forEach()` und `map()` klar unterscheiden.

1. Was ist ein Callback?

Ein Callback ist eine Funktion, die einer anderen Funktion als Argument übergeben wird. Die empfangende Funktion entscheidet, wann sie den Callback ausführt.

```
function begruessen(name) {
  console.log(`Hallo ${name}!`);
}

function verarbeiteName(name, callback) {
  callback(name);
}

verarbeiteName("Mara", begruessen);
// Ausgabe: Hallo Mara!
```

Funktion vs. Funktionsaufruf

Schreibweise	Bedeutung
begruessen	Die Funktion selbst wird übergeben. Das ist der Callback.
begruessen()	Die Funktion wird sofort ausgeführt; ihr Rückgabewert wird übergeben.

Merksatz

Ohne Klammern übergibst du die Funktion. Mit Klammern führst du sie aus.

Callbacks mit mehreren Funktionen

Dieselbe Higher-Order Function kann unterschiedliches Verhalten erhalten:

```
function gross(text) { return text.toUpperCase(); }
function klein(text) { return text.toLowerCase(); }

function transformiere(text, callback) {
  return callback(text);
}

console.log(transformiere("Hallo", gross)); // HALLO
console.log(transformiere("Hallo", klein)); // hallo
```

2. Callback-Schreibweisen

Benannte Funktion

```
function zeigeSkill(skill) {  
  console.log(skill);  
}  
  
skills.forEach(zeigeSkill);
```

Praktisch, wenn die Funktion mehrfach gebraucht wird oder einen sprechenden Namen verdient.

Anonyme Funktion

```
skills.forEach(function (skill) {  
  console.log(skill);  
});
```

Die Funktion hat keinen Namen und steht direkt an der Stelle, an der sie gebraucht wird.

Arrow Function

```
skills.forEach((skill) => {  
  console.log(skill);  
});  
  
// Kurzform bei genau einem Ausdruck:  
skills.forEach(skill => console.log(skill));
```

Kurzform

Bei genau einem Parameter sind runde Klammern optional. Bei genau einem Ausdruck sind geschweifte Klammern und return optional.

Rückgabewert nicht vergessen

```
const laengen = skills.map(skill => skill.length);  
  
// Mit Block muss return ausdrücklich stehen:  
const laengen2 = skills.map(skill => {  
  return skill.length;  
});
```

3. Higher-Order Functions

Eine Higher-Order Function ist eine Funktion, die mindestens eine Funktion als Argument erhält oder eine Funktion zurückgibt. Viele JavaScript-Array-Methoden sind Higher-Order Functions.

Denkmodell

Die Methode steuert den Ablauf; dein Callback beschreibt, was bei jedem Element geprüft, berechnet oder ausgeführt werden soll.

map() - jedes Element umwandeln

Liefert ein neues Array gleicher Länge. Der Rückgabewert des Callbacks wird zum neuen Element.

```
const zahlen = [1, 2, 3];
const doppelt = zahlen.map(zahl => zahl * 2);
// [2, 4, 6]
```

filter() - Elemente auswählen

Liefert ein neues Array mit allen Elementen, für die der Callback true ergibt.

```
const skills = ["HTML", "CSS", "JavaScript", "React"];
const lang = skills.filter(skill => skill.length > 4);
// ["JavaScript", "React"]
```

some() - gibt es mindestens einen Treffer?

Liefert einen Boolean und stoppt, sobald ein Treffer gefunden wurde.

```
const hatReact = skills.some(skill => skill === "React");
// true
```

find() - ersten Treffer holen

Liefert das erste passende Element oder undefined.

```
const ersterLangerSkill = skills.find(skill => skill.length > 4);
// "JavaScript"
```

4. forEach() und Events

forEach() - für jedes Element etwas tun

forEach() führt den Callback einmal pro Element aus. Typisch sind Ausgaben, DOM-Änderungen oder andere Seiteneffekte. Die Methode liefert kein neues Array.

```
const skills = ["HTML", "CSS", "JavaScript"];

skills.forEach((skill, index) => {
  console.log(`${index + 1}. ${skill}`);
});
// 1. HTML
// 2. CSS
// 3. JavaScript
```

addEventListener() - Callback bei einem Ereignis

Auch Event Listener arbeiten mit Callbacks: Der Browser ruft die Funktion erst auf, wenn das Ereignis eintritt.

```
const button = document.querySelector("button");

button.addEventListener("click", event => {
  console.log("Button wurde geklickt");
  console.log(event.target);
});
```

Wichtig

Schreibe handleClick statt handleClick(), wenn der Browser die Funktion erst beim Klick ausführen soll.

```
function handleClick() {
  console.log("Klick!");
}

button.addEventListener("click", handleClick); // korrekt
```

Typische Callback-Parameter

Array-Methoden können dem Callback (element, index, array) geben. Du musst nur die Parameter angeben, die du tatsächlich brauchst.

5. Vergleich: forEach() und map()

Frage	forEach()	map()
Zweck	Aktion pro Element	Elemente umwandeln
Rückgabewert	undefined	neues Array
Ergebnislänge	kein Ergebnis-Array	gleich wie Ausgangsarray
Typisch	loggen, DOM ändern	Werte berechnen/formatierten

```
const namen = ["Mia", "Noah"];

const ergebnis1 = namen.forEach(name => console.log(name));
console.log(ergebnis1); // undefined

const ergebnis2 = namen.map(name => name.toUpperCase());
console.log(ergebnis2); // ["MIA", "NOAH"]
```

Entscheidungsfrage

Brauchst du ein neues Array? Nimm map(). Willst du nur eine Aktion für jedes Element ausführen? Nimm forEach().

Häufiger Fehler

```
// Das funktioniert nicht wie gewünscht:
const neu = namen.forEach(name => name.toUpperCase());
// neu ist undefined

// So ist es richtig:
const neu2 = namen.map(name => name.toUpperCase());
```

6. filter() + map() kombinieren

Methoden können verkettet werden. Das Ergebnis der ersten Methode wird zum Ausgangspunkt der nächsten.

```
const skills = ["HTML", "CSS", "JavaScript", "React", "Git", "TypeScript"];

const labels = skills
  .filter(skill => skill.length > 4)
  .map(skill => `Skill: ${skill}`);

console.log(labels);
// ["Skill: JavaScript", "Skill: React", "Skill: TypeScript"]
```

Ablauf

1 · filter	2 · map	3 · Ergebnis
nur Länge > 4	Text voranstellen	neues Array

Leserichtung

Lies eine Methodenkette von oben nach unten: auswählen → umwandeln → verwenden.

Zwischenergebnisse zum Debuggen

```
const langeSkills = skills.filter(skill => skill.length > 4);
console.log(langeSkills);

const labels = langeSkills.map(skill => `Skill: ${skill}`);
console.log(labels);
```

Das getrennte Schreiben ist länger, aber beim Lernen und Debuggen oft leichter nachzuvollziehen.

7. Abschlussübung

Aufgabe

Nimm das Skill-Array. Wähle alle Skills mit mehr als vier Zeichen aus, formatiere sie als Skill: NAME und gib anschließend jeden Eintrag einzeln aus.

```
const mySkills = [
  "HTML",
  "CSS",
  "JavaScript",
  "React",
  "Git",
  "TypeScript"
];

mySkills
  .filter(skill => skill.length > 4)
  .map(skill => `Skill: ${skill}`)
  .forEach(skill => {
    console.log(skill);
  });
```

Ausgabe

```
Skill: JavaScript
Skill: React
Skill: TypeScript
```

Was passiert hier?

1. `filter()` behält JavaScript, React und TypeScript.
2. `map()` erzeugt aus jedem Namen einen formatierten String.
3. `forEach()` gibt jeden fertigen String aus.

Saubere Lösung

Im `map()`-Callback musst du `skill` nicht neu zuweisen. Gib den neuen String direkt zurück: `skill => `Skill: ${skill}``.

8. Kurzreferenz

Methode	Frage	Ergebnis
map	Wie wird jedes Element verändert?	neues Array
filter	Welche Elemente bleiben?	neues Array
some	Gibt es mindestens einen Treffer?	Boolean
find	Was ist der erste Treffer?	Element / undefined
forEach	Was soll pro Element passieren?	undefined

Die wichtigsten Merksätze

1
Ein Callback wird als Funktion übergeben - normalerweise ohne ().

2
Higher-Order Functions nehmen Funktionen entgegen oder geben Funktionen zurück.

3
filter wählt aus, map verändert, forEach führt eine Aktion aus.

4
Arrow Functions sind besonders praktisch für kurze Callbacks.

Mini-Check

Kannst du erklären, warum `button.addEventListener("click", handleClick)` korrekt ist? Kannst du vorhersagen, was eine Kette aus `filter().map().forEach()` in jedem Schritt erzeugt? Dann sitzt Tag 16.

Weiter geht's mit dem nächsten Frontend-Lerntag.